

Otten Coffee

#MengopikanIndonesia

TECHNICAL TEST — SUBMISSION

Test 3

Analytics Engineering & Pipeline Design

dbt Layer Architecture · Airflow Orchestration · ELT Pipeline · Data Quality

Dataset Brazilian E-Commerce (Olist) — Public PostgreSQL

Role BI Engineer

Author Nur Imam Masri

Date May 2026

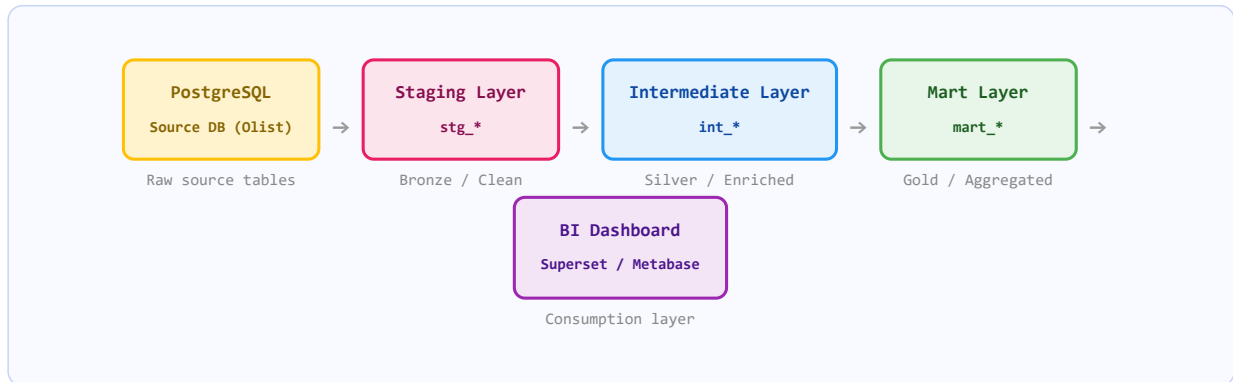
Contents

#	Section
1	Architecture Overview — Layered ELT Design
2	Staging Layer (stg_*) — Model Design Decisions
3	Intermediate Layer (int_*) — Business Logic & Enrichment
4	Mart Layer (mart_*) — Aggregated Outputs for BI
	4.5 Full SQL — Mart Models 4.6 Schema & Data Verification (DBever)
5	Loading Strategy — Incremental vs Full Refresh
6	Orchestration Tool — Choice & Justification
7	Data Quality — Checks at Every Layer
8	Schema Change Handling
9	Apache Airflow — Full ELT Pipeline Implementation
	9.4 Airflow UI Screenshots 9.5 Full DAG Code 9.6–9.9 Cosmos · Docker · Setup · Monitoring

1 Architecture Overview — Layered ELT Design

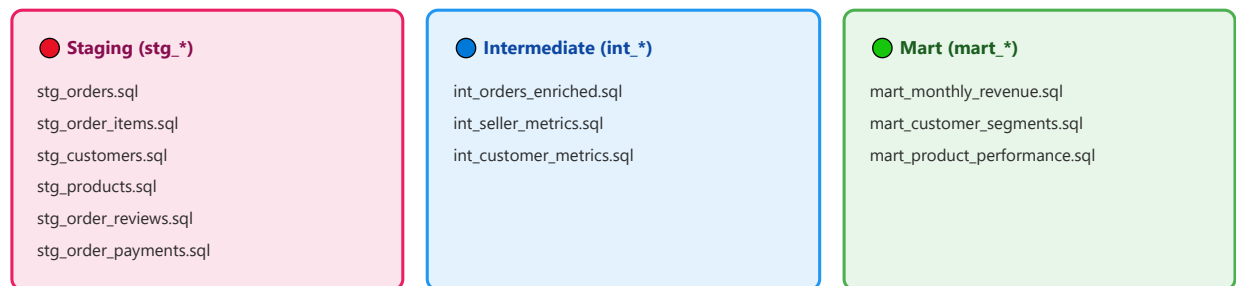
Staging → Intermediate → Mart: separating concerns across transformation layers

1.1 End-to-End Pipeline



1.2 Why This Three-Layer Architecture?

Each layer has a single, clearly bounded responsibility. This separation yields four concrete benefits: **debuggability** (isolate failures to one layer), **reusability** (multiple mart models can share one intermediate model), **testability** (data quality checks can be scoped per layer), and **maintainability** (changing business logic in one model does not break unrelated models).



1.3 Lineage Graph (Model Dependencies)

SOURCE	STAGING	INTERMEDIATE	MART
orders	→ stg_orders	→ int_orders_enriched	→ mart_monthly_revenue
customers	→ stg_customers	→ int_orders_enriched	→ mart_customer_segments
order_items	→ stg_order_items	→ int_orders_enriched	→ mart_product_performance
order_payments	→ stg_order_payments	→ int_orders_enriched	
order_reviews	→ stg_order_reviews	→ int_orders_enriched	
products + translation	→ stg_products	→ int_seller_metrics → int_customer_metrics	

DESIGN PRINCIPLE: NO STAGING → MART SHORTCUTS

All mart models consume from intermediate models, never directly from staging. This prevents business logic from being duplicated across multiple mart models and ensures that the intermediate layer is the single source of truth for enriched metrics.

2 Staging Layer (stg_*)

Clean mirror of source — rename, cast, null-handle. Zero business logic.

2.1 Design Rules Applied to All Staging Models

Rule	Implementation	Rationale
Rename columns	Source names → clear snake_case (e.g., <code>order_purchase_timestamp</code> → <code>purchased_at</code>)	Downstream models reference clean names; source typos are fixed once here
Explicit type casts	<code>price::numeric(10,2)</code> , <code>review_score::int</code>	Prevents implicit cast errors in downstream aggregations; makes schema self-documenting
Null filtering	<code>WHERE order_id IS NOT NULL</code>	Orphaned records without PKs would corrupt downstream joins
No business logic	No CASE WHEN, no joins between source tables (except <code>stg_products</code>)	Keeps staging as a stable, predictable foundation; business logic changes belong in int_*
Empty string → NULL	<code>NULLIF(TRIM(review_comment_title), '')</code>	Standardises absence of data; avoids " vs NULL inconsistency in analytics

2.2 Notable Design Decisions Per Model

stg_products — Exception: One JOIN Allowed

Why stg_products joins the translation table

The `product_category_name_translation` table is a pure reference/lookup table with no business semantics — it simply maps Portuguese names to English equivalents. Including this join in staging eliminates the need to replicate it in every downstream model that needs English category names. A `LEFT JOIN` is used (not INNER) to preserve products with untranslated categories, which fall back to `'uncategorized'` via COALESCE.

Exception criteria: A staging JOIN is acceptable when the joined table is a static reference/lookup (no business logic, rarely changes, no PK ambiguity).

stg_products — Typo Fix in Source Column Names

```
-- Source has typos: "lenght" instead of "length" (missing 't')
p.product_name_lenght      AS product_name_length,
p.product_description_lenght AS description_length
```

The original Olist dataset has misspelled column names. Staging is the **only** place where these are corrected — all downstream models use the clean `_length` spelling.

stg_order_reviews — Derived boolean: is_low_score

```
(review_score <= 2) AS is_low_score
```

This is the only derived column in staging. It is placed here because: (1) it is a simple, non-contextual rule ($\text{score} \leq 2 = \text{poor}$), (2) it will be consumed by multiple downstream models (`int_orders_enriched` , `mart_product_performance`), and (3) it requires no joins to compute. If the threshold were to change, it would only need updating in one place.

stg_order_payments — Keeps Raw Payment Grain

DESIGN NOTE: ONE ROW PER PAYMENT RECORD

A single order can have multiple payment rows (e.g., credit card + voucher, or instalments recorded separately). `stg_order_payments` preserves this grain. Aggregation to order-level (total_payment, primary_payment_type) happens in `int_orders_enriched` , where the business context of "per-order payment" is appropriate.

2.3 Complete Model Summary

Model	Source	Key Transformations	PK
stg_orders	orders	Rename all timestamp cols to *_at; status kept as-is	order_id
stg_order_items	order_items	Cast price/freight to NUMERIC(10,2); add total_item_cost	order_id + item_sequence
stg_customers	customers	INITCAP city; UPPER state; rename zip_code	customer_id
stg_products	products + translation	LEFT JOIN translation; fix typo columns; COALESCE category	product_id
stg_order_reviews	order_reviews	Cast score; derive is_low_score boolean; NULLIF blanks	(none — source has no PK)
stg_order_payments	order_payments	Cast payment_value to NUMERIC(10,2); filter negative values	order_id + payment_seq

2.4 Full SQL — Staging Models

All 6 staging models listed below. Materialized as `VIEW` — always fresh, zero storage cost.

stg_orders.sql

```
-- Model: stg_orders | Source: public.orders | Grain: one row per order_id

SELECT
  order_id,
  customer_id,
  order_status,
  order_purchase_timestamp      AS purchased_at,
  order_approved_at            AS approved_at,
  order_delivered_carrier_date  AS shipped_at,
  order_delivered_customer_date AS delivered_at,
  order_estimated_delivery_date AS estimated_delivery_at
FROM orders
WHERE order_id IS NOT NULL;
```

stg_customers.sql

```
-- Model: stg_customers | Source: public.customers | Grain: one row per customer_id
-- Note: customer_id is per-order; customer_unique_id is the true cross-order identity

SELECT
  customer_id,
  customer_unique_id,
  customer_zip_code_prefix      AS zip_code,
  INITCAP(LOWER(customer_city)) AS city,
  UPPER(customer_state)         AS state
FROM customers
WHERE customer_id IS NOT NULL;
```

stg_order_items.sql

```
-- Model: stg_order_items | Source: public.order_items | Grain: one row per (order_id, item_sequence)

SELECT
  order_id,
  order_item_id                AS item_sequence,
  product_id,
  seller_id,
  shipping_limit_date           AS shipping_deadline,
  ROUND(price::numeric, 2)      AS item_price,
  ROUND(freight_value::numeric, 2) AS freight_cost,
  ROUND((price + freight_value)::numeric, 2) AS total_item_cost
FROM order_items
WHERE order_id IS NOT NULL
  AND product_id IS NOT NULL
  AND price IS NOT NULL;
```

stg_order_payments.sql

```
-- Model: stg_order_payments | Source: public.order_payments
-- Note: One order can have multiple payment rows (split payments / installments)

SELECT
  order_id,
  payment_sequential            AS payment_seq,
  payment_type,
  payment_installments          AS installments,
  ROUND(payment_value::numeric, 2) AS payment_amount
FROM order_payments
```

```
WHERE order_id      IS NOT NULL
      AND payment_value IS NOT NULL
      AND payment_value >= 0;
```

stg_order_reviews.sql

```
-- Model: stg_order_reviews | Source: public.order_reviews
-- Business rule: is_low_score = TRUE when score <= 2 (1-star and 2-star = poor experience)

SELECT
    review_id,
    order_id,
    review_score::int                AS score,
    (review_score <= 2)              AS is_low_score,
    NULLIF(TRIM(review_comment_title), '') AS review_title,
    NULLIF(TRIM(review_comment_message), '') AS review_message,
    review_creation_date             AS created_at,
    review_answer_timestamp          AS answered_at
FROM order_reviews
WHERE order_id      IS NOT NULL
      AND review_score IS NOT NULL;
```

stg_products.sql

```
-- Model: stg_products | Source: public.products + public.product_category_name_translation
-- Exception: LEFT JOIN to translation table allowed (static reference, no business logic)
-- Also fixes source typos: product_name_lenght → product_name_length

SELECT
    p.product_id,
    COALESCE(t.product_category_name_english, 'uncategorized') AS category_name_en,
    p.product_category_name                                     AS category_name_pt,
    p.product_name_lenght                                     AS product_name_length,
    p.product_description_lenght                             AS description_length,
    p.product_photos_qty                                     AS photos_count,
    ROUND(p.product_weight_g::numeric, 0)                   AS weight_grams,
    ROUND(p.product_length_cm::numeric, 1)                   AS length_cm,
    ROUND(p.product_height_cm::numeric, 1)                   AS height_cm,
    ROUND(p.product_width_cm::numeric, 1)                    AS width_cm
FROM products p
LEFT JOIN product_category_name_translation t
    ON t.product_category_name = p.product_category_name
WHERE p.product_id IS NOT NULL;
```

3 Intermediate Layer (int_*)

Business logic, cross-model joins, and derived metrics. The "facts" of the warehouse.

3.1 Design Philosophy

The intermediate layer is where domain knowledge lives. Models here answer the question: *"What does the business need to know about each entity, combining all relevant context?"* Each intermediate model has a clearly defined grain (one row per order, seller, or customer) and provides reusable, pre-computed metrics that multiple mart models can consume.

INTERMEDIATE MODELS ARE NEVER QUERIED DIRECTLY BY BI TOOLS.

They are stepping stones — too granular for a Senior Manager dashboard, but too aggregated for a staging model. Their purpose is to enable mart models to be simple SELECT + GROUP BY operations.

3.2 int_orders_enriched — The Central Fact Model

Grain: One row per order_id

Sources joined: stg_orders ⋈ stg_customers ⋈ stg_order_items (aggregated) ⋈ stg_order_payments (aggregated) ⋈ stg_order_reviews (aggregated)

Key Derived Metrics

Column	Formula	Business Meaning
delivery_delay_days	<code>EPOCH(delivered_at - estimated_delivery_at) / 86400</code>	Positive = late by N days. Negative = early. NULL = not yet delivered. Rounds to 1 decimal for precision without noise.
is_late_delivery	<code>delivered_at > estimated_delivery_at</code>	Boolean flag for easy COUNT/SUM in aggregations downstream. Avoids repeating the date comparison in every mart model.
total_delivery_days	<code>EPOCH(delivered_at - purchased_at) / 86400</code>	End-to-end fulfilment time from customer perspective. Different from delay_days which measures against the promise.
primary_payment_type	Payment type with highest payment_amount in order	Identifies dominant payment method for orders split across multiple payment types.
has_low_score	<code>BOOL_OR(is_low_score)</code> from review_agg	TRUE if any review for this order is 1–2 stars. Used as a quality flag in product and seller analytics.

Why Use CTEs Within the Model (not subqueries)?

```
WITH order_agg AS (  
    SELECT order_id, SUM(item_price) AS total_items_price ...  
    FROM stg_order_items GROUP BY order_id  
),  
payment_agg AS ( ... ),  
review_agg AS ( ... )  
SELECT o.*, c.*, oa.*, pa.*, ra.*, -- delivery metrics ...  
FROM stg_orders o  
JOIN stg_customers c ON ...  
LEFT JOIN order_agg oa ON oa.order_id = o.order_id ...
```

Each upstream model (order_items, payments, reviews) is aggregated to order-grain in its own CTE *before* joining to orders. This prevents the fan-out problem — if orders were joined to non-aggregated items first, each order would appear N times (once per item), causing inflated aggregations downstream.

3.3 int_seller_metrics — Operational Performance per Seller

Grain: One row per seller_id

Aggregates all completed orders containing items from each seller. Provides: revenue, volume, customer satisfaction (avg review score, low-score rate), and delivery performance (late rate, avg days late). Also identifies each seller's top category by revenue using `DISTINCT ON`.

3.4 int_customer_metrics — Lifetime Behaviour per Customer

Grain: One row per customer_unique_id

Derives RFM-adjacent metrics: total orders, total spend, first/last order dates, customer lifetime days, avg days between purchases (frequency), and preferred state. These feed directly into `mart_customer_segments`.

`avg_days_between_orders` uses $(lifetime_days) / (total_orders - 1)$ to measure true inter-purchase intervals, and returns NULL for one-time buyers (where the denominator would be 0).

3.5 Full SQL — int_orders_enriched.sql

Central fact model — one row per order, consumed by all mart models. Pre-aggregates items, payments, and reviews to order grain before joining to prevent fan-out.

```
-- Model: int_orders_enriched | Grain: one row per order_id
-- Sources: stg_orders, stg_customers, stg_order_items, stg_order_payments, stg_order_reviews

WITH order_agg AS (
    SELECT
        order_id,
        ROUND(SUM(item_price)::numeric, 2) AS total_items_price,
        ROUND(SUM(freight_cost)::numeric, 2) AS total_freight,
        COUNT(item_sequence) AS item_count
    FROM stg_order_items
    GROUP BY order_id
),
payment_agg AS (
    SELECT
        order_id,
        ROUND(SUM(payment_amount)::numeric, 2) AS total_payment,
        MAX(installments) AS max_installments,
        (
            -- Dominant payment type = highest individual payment amount
            SELECT payment_type
            FROM stg_order_payments sp2
            WHERE sp2.order_id = sp.order_id
            ORDER BY payment_amount DESC
            LIMIT 1
        ) AS primary_payment_type
    FROM stg_order_payments sp
    GROUP BY order_id
),
review_agg AS (
    SELECT
        order_id,
        ROUND(AVG(score)::numeric, 1) AS avg_review_score,
        BOOL_OR(is_low_score) AS has_low_score,
        COUNT(*) AS review_count
    FROM stg_order_reviews
    GROUP BY order_id
)
SELECT
    o.order_id,
    o.customer_id,
    c.customer_unique_id,
    c.city AS customer_city,
    c.state AS customer_state,
    o.order_status,
    o.purchased_at, o.approved_at, o.shipped_at,
    o.delivered_at, o.estimated_delivery_at,
    COALESCE(oa.total_items_price, 0) AS total_items_price,
    COALESCE(oa.total_freight, 0) AS total_freight,
    COALESCE(oa.item_count, 0) AS item_count,
    COALESCE(pa.total_payment, 0) AS total_payment,
    pa.primary_payment_type,
    COALESCE(pa.max_installments, 1) AS max_installments,
    ra.avg_review_score,
    COALESCE(ra.has_low_score, FALSE) AS has_low_score,
    COALESCE(ra.review_count, 0) AS review_count,

    -- Delivery performance (positive = late, negative = early, NULL = not yet delivered)
    CASE
        WHEN o.delivered_at IS NOT NULL AND o.estimated_delivery_at IS NOT NULL
        THEN ROUND(EXTRACT(EPOCH FROM (o.delivered_at - o.estimated_delivery_at)) / 86400.0, 1)
    END AS delivery_delay_days,

    CASE WHEN o.delivered_at > o.estimated_delivery_at THEN TRUE ELSE FALSE
    END AS is_late_delivery,
```

```
CASE
    WHEN o.delivered_at IS NOT NULL AND o.purchased_at IS NOT NULL
    THEN ROUND(EXTRACT(EPOCH FROM (o.delivered_at - o.purchased_at)) / 86400.0, 1)
    END AS total_delivery_days

FROM stg_orders o
JOIN stg_customers c    ON c.customer_id = o.customer_id
LEFT JOIN order_agg oa  ON oa.order_id   = o.order_id
LEFT JOIN payment_agg pa ON pa.order_id   = o.order_id
LEFT JOIN review_agg ra ON ra.order_id    = o.order_id;
```

3.6 Full SQL — int_customer_metrics.sql & int_seller_metrics.sql

int_customer_metrics.sql

One row per `customer_unique_id` . Derives lifetime spend, purchase frequency, and preferred state. Feeds `mart_customer_segments` .

```
-- Model: int_customer_metrics | Grain: one row per customer_unique_id
-- Excludes cancelled orders

WITH customer_base AS (
    SELECT customer_unique_id, order_id, purchased_at,
           total_payment, avg_review_score, order_status, customer_state
    FROM int_orders_enriched
    WHERE order_status != 'canceled' AND purchased_at IS NOT NULL
),
customer_state_mode AS (
    -- Most frequent state per customer (DISTINCT ON picks top by window count)
    SELECT DISTINCT ON (customer_unique_id)
           customer_unique_id,
           customer_state AS preferred_state,
           COUNT(*) OVER (PARTITION BY customer_unique_id, customer_state) AS state_orders
    FROM customer_base
    ORDER BY customer_unique_id, state_orders DESC
)
SELECT
    cb.customer_unique_id,
    COUNT(DISTINCT cb.order_id) AS total_orders,
    ROUND(SUM(cb.total_payment)::numeric, 2) AS total_spend,
    ROUND(AVG(cb.total_payment)::numeric, 2) AS avg_order_value,
    MIN(cb.purchased_at) AS first_order_at,
    MAX(cb.purchased_at) AS last_order_at,
    ROUND(EXTRACT(DAY FROM (MAX(cb.purchased_at) - MIN(cb.purchased_at)))::numeric, 0) AS customer_lifetime_days,
    -- NULL for single-purchase customers (avoid divide-by-zero)
    CASE WHEN COUNT(DISTINCT cb.order_id) > 1
         THEN ROUND(EXTRACT(DAY FROM (MAX(cb.purchased_at) - MIN(cb.purchased_at)))
                    / (COUNT(DISTINCT cb.order_id) - 1.0), 0)
         END AS avg_days_between_orders,
    ROUND(AVG(cb.avg_review_score)::numeric, 2) AS avg_review_score,
    csm.preferred_state
FROM customer_base cb
LEFT JOIN customer_state_mode csm ON csm.customer_unique_id = cb.customer_unique_id
GROUP BY cb.customer_unique_id, csm.preferred_state
ORDER BY total_spend DESC;
```

int_seller_metrics.sql

One row per `seller_id` . Combines revenue, satisfaction, and delivery reliability. Top category identified via `DISTINCT ON` .

```
-- Model: int_seller_metrics | Grain: one row per seller_id

WITH seller_base AS (
    SELECT oi.seller_id, oi.order_id, oi.item_price,
           oe.order_status, oe.avg_review_score,
           oe.is_late_delivery, oe.delivery_delay_days, oe.has_low_score
    FROM stg_order_items oi
    JOIN int_orders_enriched oe ON oe.order_id = oi.order_id
    WHERE oe.order_status != 'canceled'
),
seller_top_category AS (
    SELECT DISTINCT ON (oi.seller_id)
           oi.seller_id,
           p.category_name_en AS top_category,
           SUM(oi.item_price) OVER (PARTITION BY oi.seller_id, p.category_name_en) AS category_revenue
    FROM stg_order_items oi
    JOIN stg_products p ON p.product_id = oi.product_id
    JOIN int_orders_enriched oe ON oe.order_id = oi.order_id
    WHERE oe.order_status != 'canceled'
)
```

```

ORDER BY oi.seller_id, category_revenue DESC
)
SELECT
    sb.seller_id,
    COUNT(DISTINCT sb.order_id)                AS total_orders,
    COUNT(*)                                    AS total_items_sold,
    ROUND(SUM(sb.item_price)::numeric, 2)       AS total_revenue,
    ROUND(AVG(sb.item_price)::numeric, 2)       AS avg_item_price,
    ROUND(SUM(sb.item_price)::numeric / NULLIF(COUNT(DISTINCT sb.order_id),0), 2)
                                                AS avg_revenue_per_order,
    ROUND(AVG(sb.avg_review_score)::numeric, 2) AS avg_review_score,
    COUNT(CASE WHEN sb.has_low_score THEN 1 END) AS low_score_orders,
    ROUND(COUNT(CASE WHEN sb.has_low_score THEN 1 END) * 100.0
          / NULLIF(COUNT(DISTINCT sb.order_id),0), 1) AS low_score_rate_pct,
    COUNT(CASE WHEN sb.is_late_delivery THEN 1 END) AS late_delivery_count,
    ROUND(COUNT(CASE WHEN sb.is_late_delivery THEN 1 END) * 100.0
          / NULLIF(COUNT(DISTINCT sb.order_id),0), 1) AS late_delivery_rate_pct,
    ROUND(AVG(CASE WHEN sb.is_late_delivery THEN sb.delivery_delay_days END)::numeric, 1)
                                                AS avg_days_late,
    stc.top_category
FROM seller_base sb
LEFT JOIN seller_top_category stc ON stc.seller_id = sb.seller_id
GROUP BY sb.seller_id, stc.top_category
ORDER BY total_revenue DESC;

```

4 Mart Layer (mart_*)

Business-ready aggregations — directly consumed by BI dashboards and Senior Manager reports

4.1 Design Philosophy

Mart models are designed for **BI consumption**: wide, flat, pre-aggregated tables that a Senior Manager or a BI tool can query without complex joins. Each mart model answers a specific business question, and column names are chosen for clarity over technical precision.

4.2 mart_monthly_revenue

Business Question: "How is our revenue performing month-on-month?"

Column	Type	Description
month	date	First day of the calendar month
total_revenue	numeric	Sum of all payment values for the month
total_orders	int	Count of distinct orders placed
unique_customers	int	Distinct customer_unique_ids who ordered
avg_order_value	numeric	Revenue / orders for the month
mom_revenue_growth_pct	numeric	% change vs prior month revenue
ytd_revenue	numeric	Cumulative revenue within the calendar year
late_delivery_rate_pct	numeric	% of orders delivered after promise date

4.3 mart_customer_segments

Business Question: "Who are our customers and how do we retain them?"

Classifies every customer into one of four segments based on purchase recency and frequency. Designed to drive CRM targeting and retention campaign prioritisation.

Segment	Rule	Count (dataset)	Avg Spend (R\$)
new	Exactly 1 order ever	92,635	161.56
returning	2+ orders, last order ≤ 180 days ago	1,239	321.31
at_risk	2+ orders, last order 181–365 days ago	1,136	303.39
churned	2+ orders, last order > 365 days ago	549	285.31

CRITICAL BUSINESS INSIGHT FROM SEGMENTATION

96.8% of customers (92,635 out of ~95,500) made only one purchase — they are classified as "new" indefinitely. Returning customers spend

2× MORE

(R\$ 321 vs R\$ 162) than one-time buyers. This makes retention the highest-ROI business initiative: converting even 1% of new customers to returning adds ~R\$ 15M in incremental revenue.

Why use dataset max date (not NOW()) as reference?

The segmentation uses `MAX(purchased_at)` from the dataset as the reference date instead of `CURRENT_TIMESTAMP`. This makes the model **reproducible**: running the same query twice at different times produces the same result for historical analysis. Using `NOW()` would continuously re-classify customers as time passes, making historical snapshots inconsistent.

4.4 mart_product_performance

Business Question: "Which products drive revenue and which need attention?"

Provides product-level metrics combining commercial performance (revenue, units sold) with quality signals (review scores, low-score rates, late delivery rates). Includes three ranking dimensions:

- `revenue_rank` — global rank across all products
- `category_revenue_rank` — rank within the product's category (enables "best in category" analysis)
- `quality_adjusted_rank` — ranks by `avg_review_score × total_revenue`, penalising high-revenue products with poor ratings

The quality-adjusted ranking is designed to surface products that are commercially important but risk eroding brand trust — a signal for Senior Managers to investigate product quality issues.

4.5 Full SQL — Mart Models

Complete SQL for all three mart models. Materialized as **tables** with full refresh strategy — optimized for BI consumption.

mart_monthly_revenue.sql

```
-- =====
-- Model : mart_monthly_revenue | Layer: Mart
-- Sources: int_orders_enriched
-- Grain : one row per calendar month
-- Refresh: full refresh (aggregations are non-idempotent with incremental)
-- =====

WITH monthly_base AS (
  SELECT
    DATE_TRUNC('month', purchased_at)::date AS month,
    COUNT(DISTINCT order_id) AS total_orders,
    COUNT(DISTINCT customer_unique_id) AS unique_customers,
    ROUND(SUM(total_payment)::numeric, 2) AS total_revenue,
    ROUND(AVG(total_payment)::numeric, 2) AS avg_order_value,
    ROUND(SUM(total_items_price)::numeric, 2) AS total_items_revenue,
    ROUND(SUM(total_freight)::numeric, 2) AS total_freight_revenue,
    COUNT(CASE WHEN is_late_delivery THEN 1 END) AS late_delivery_count,
    ROUND(
      COUNT(CASE WHEN is_late_delivery THEN 1 END) * 100.0
      / NULLIF(COUNT(DISTINCT order_id), 0)
      , 1) AS late_delivery_rate_pct,
    ROUND(AVG(avg_review_score)::numeric, 2) AS avg_review_score
  FROM int_orders_enriched
  WHERE order_status NOT IN ('canceled', 'unavailable')
    AND purchased_at IS NOT NULL
  GROUP BY 1
)
SELECT
  month, total_orders, unique_customers, total_revenue, avg_order_value,
  total_items_revenue, total_freight_revenue,
  late_delivery_count, late_delivery_rate_pct, avg_review_score,

  -- Month-over-month comparisons
  LAG(total_revenue) OVER (ORDER BY month) AS prev_month_revenue,
  LAG(total_orders) OVER (ORDER BY month) AS prev_month_orders,
  ROUND(
    (total_revenue - LAG(total_revenue) OVER (ORDER BY month)) * 100.0
    / NULLIF(LAG(total_revenue) OVER (ORDER BY month), 0)
    , 2) AS mom_revenue_growth_pct,
  ROUND(
    (total_orders - LAG(total_orders) OVER (ORDER BY month)) * 100.0
    / NULLIF(LAG(total_orders) OVER (ORDER BY month), 0)
    , 2) AS mom_orders_growth_pct,

  -- Running totals (year-to-date)
  SUM(total_revenue) OVER (
    PARTITION BY EXTRACT(YEAR FROM month)
    ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS ytd_revenue,
  SUM(total_orders) OVER (
    PARTITION BY EXTRACT(YEAR FROM month)
    ORDER BY month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS ytd_orders
FROM monthly_base
ORDER BY month;
```


mart_customer_segments.sql

```
-- =====
-- Model : mart_customer_segments | Layer: Mart
-- Sources: int_customer_metrics, int_orders_enriched
-- Grain : one row per customer_unique_id
-- Note : Uses MAX(purchased_at) as reference date – reproducible, not NOW()
-- Segments: new (1 order) | returning (<=180d) | at_risk (181-365d) | churned (>365d)
-- =====

WITH latest_date AS (
    SELECT MAX(purchased_at) AS reference_date
    FROM int_orders_enriched WHERE order_status != 'canceled'
),
customer_data AS (
    SELECT cm.*, ld.reference_date,
        ROUND(EXTRACT(DAY FROM (ld.reference_date - cm.last_order_at))::numeric, 0)
        AS days_since_last_order
    FROM int_customer_metrics cm CROSS JOIN latest_date ld
),
segmented AS (
    SELECT *,
        CASE
            WHEN total_orders = 1 THEN 'new'
            WHEN total_orders > 1 AND days_since_last_order <= 180 THEN 'returning'
            WHEN total_orders > 1 AND days_since_last_order BETWEEN 181 AND 365 THEN 'at_risk'
            WHEN total_orders > 1 AND days_since_last_order > 365 THEN 'churned'
            ELSE 'unknown'
        END AS customer_segment,
        NTILE(4) OVER (ORDER BY total_spend DESC) AS spend_quartile
    FROM customer_data
)
SELECT
    customer_unique_id, customer_segment, total_orders,
    ROUND(total_spend::numeric, 2) AS total_spend,
    ROUND(avg_order_value::numeric, 2) AS avg_order_value,
    first_order_at, last_order_at, days_since_last_order,
    customer_lifetime_days, avg_days_between_orders,
    ROUND(avg_review_score::numeric, 2) AS avg_review_score,
    preferred_state, spend_quartile,
    CASE spend_quartile
        WHEN 1 THEN 'high_value' WHEN 2 THEN 'mid_value'
        WHEN 3 THEN 'low_value' WHEN 4 THEN 'at_risk_value'
    END AS value_tier,
    reference_date AS reference_date_used
FROM segmented
ORDER BY total_spend DESC;
```

mart_product_performance.sql

```
-- =====
-- Model : mart_product_performance | Layer: Mart
-- Sources: stg_products, stg_order_items, int_orders_enriched
-- Grain : one row per product_id | Excludes: cancelled orders
-- Rankings: revenue_rank | category_revenue_rank | quality_adjusted_rank
-- =====

WITH product_sales AS (
    SELECT
        oi.product_id, p.category_name_en AS category,
        COUNT(DISTINCT oi.order_id) AS total_orders,
        COUNT(oi.item_sequence) AS units_sold,
        ROUND(SUM(oi.item_price)::numeric, 2) AS total_revenue,
        ROUND(AVG(oi.item_price)::numeric, 2) AS avg_unit_price,
        ROUND(SUM(oi.freight_cost)::numeric, 2) AS total_freight,
        ROUND(AVG(oe.avg_review_score)::numeric, 2) AS avg_review_score,
        COUNT(CASE WHEN oe.has_low_score THEN 1 END) AS low_score_order_count,
        ROUND(COUNT(CASE WHEN oe.has_low_score THEN 1 END) * 100.0
            / NULLIF(COUNT(DISTINCT oi.order_id), 0), 1) AS low_score_rate_pct,
        COUNT(CASE WHEN oe.is_late_delivery THEN 1 END) AS late_delivery_count,
        ROUND(COUNT(CASE WHEN oe.is_late_delivery THEN 1 END) * 100.0
            / NULLIF(COUNT(DISTINCT oi.order_id), 0), 1) AS late_delivery_rate_pct
    FROM stg_order_items oi
    JOIN stg_products p ON p.product_id = oi.product_id
    JOIN int_orders_enriched oe ON oe.order_id = oi.order_id
    WHERE oe.order_status != 'canceled'
    GROUP BY oi.product_id, p.category_name_en
),
```

```

category_totals AS (
    SELECT category, SUM(total_revenue) AS category_total_revenue
    FROM product_sales GROUP BY category
)
SELECT
    ps.product_id, ps.category, ps.total_orders, ps.units_sold,
    ps.total_revenue, ps.avg_unit_price, ps.total_freight,
    ps.avg_review_score, ps.low_score_order_count, ps.low_score_rate_pct,
    ps.late_delivery_count, ps.late_delivery_rate_pct,
    ROUND(ps.total_revenue * 100.0 / NULLIF(ct.category_total_revenue, 0), 2)
                                                AS category_revenue_share_pct,
    RANK() OVER (ORDER BY ps.total_revenue DESC) AS revenue_rank,
    RANK() OVER (PARTITION BY ps.category ORDER BY ps.total_revenue DESC)
                                                AS category_revenue_rank,
    RANK() OVER (ORDER BY (ps.avg_review_score * ps.total_revenue) DESC NULLS LAST)
                                                AS quality_adjusted_rank
FROM product_sales ps
JOIN category_totals ct ON ct.category = ps.category
ORDER BY revenue_rank;

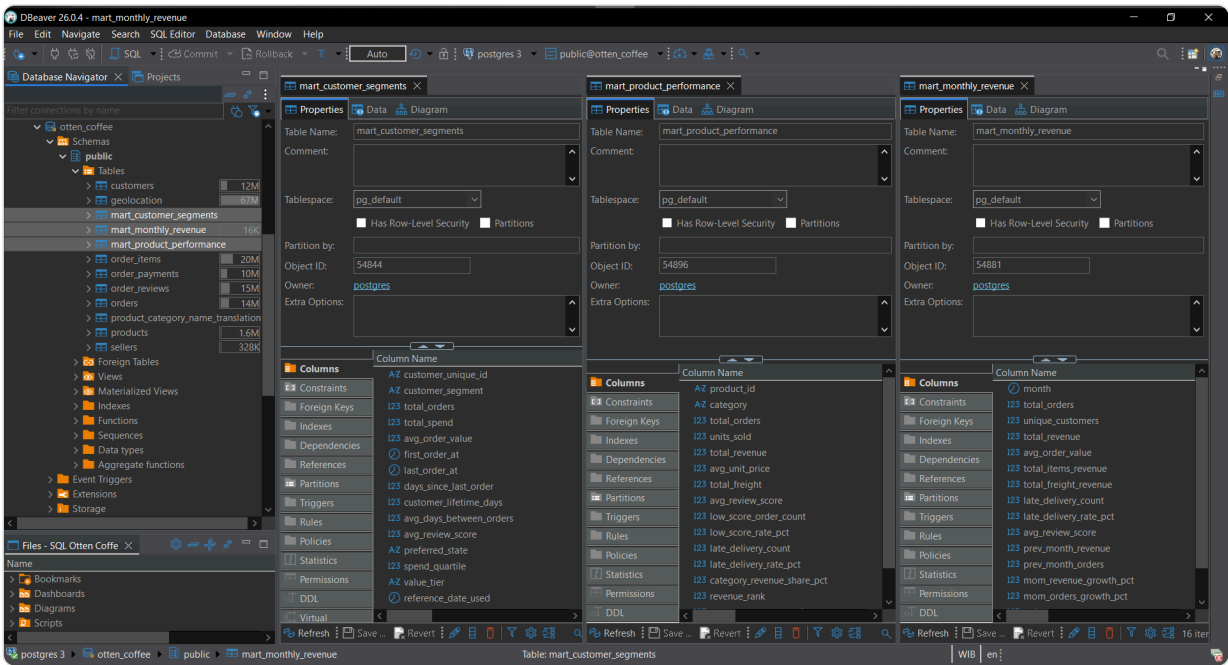
```

4.6 Mart Tables — Schema & Data Verification

Screenshots from DBeaver confirming that all three mart models were successfully materialized in PostgreSQL after running `dbt run` through Airflow.

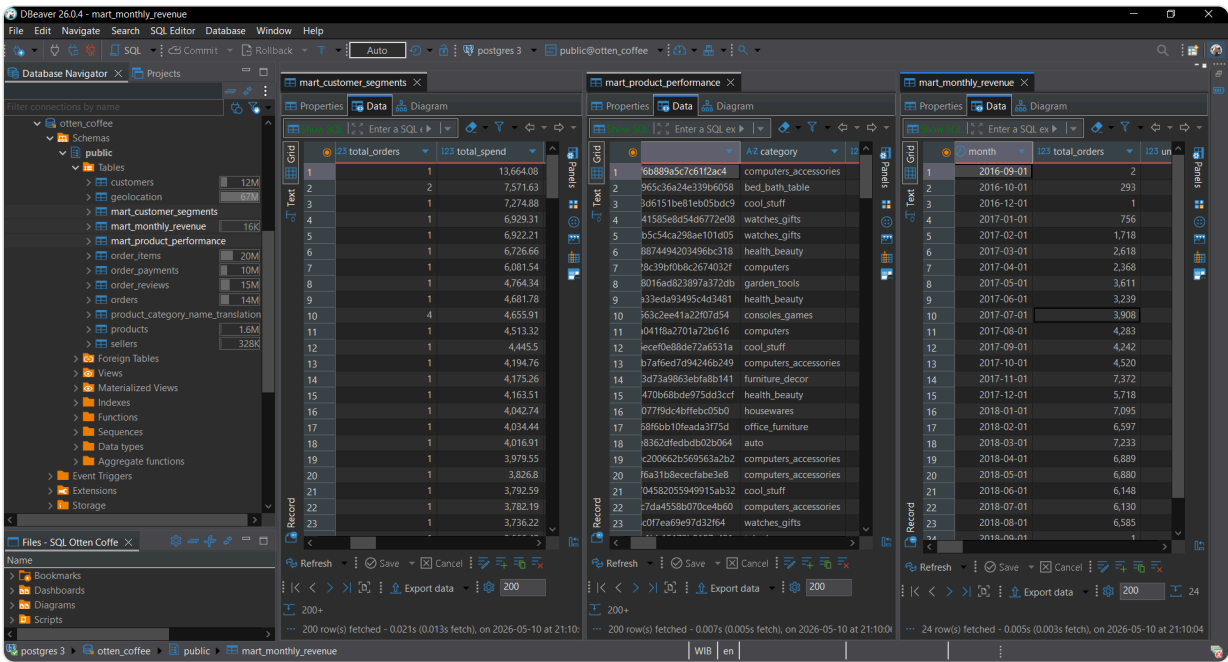
Table Schema — Column Definitions

Shows `mart_customer_segments`, `mart_product_performance`, and `mart_monthly_revenue` column definitions and data types as confirmed in the PostgreSQL database.



Mart Data — Sample Rows

Actual data rows populated in the mart tables after running the full dbt pipeline, confirming correct transformations, segment labels, and revenue figures.



5 Loading Strategy — Incremental vs Full Refresh

How raw data is loaded into each layer, and why

5.1 Strategy per Layer

Layer	Strategy	Unique Key / Watermark	Rationale
Staging stg_*	Incremental — Deduped (<code>unique_key</code> merge)	<code>order_id</code> , <code>customer_id</code> , <code>product_id</code> , etc.	New orders arrive daily; existing orders can update status (<code>processing</code> → <code>delivered</code>). Dedup-on-unique-key captures both inserts and updates without full scans.
Intermediate int_*	Incremental — Watermark (filter on <code>purchased_at</code>)	<code>purchased_at >=</code> <code>max(purchased_at) - 3</code> <code>days</code>	Derived from staging; only re-processes orders whose purchase date is within the lookback window. The 3-day buffer handles late-arriving payment or review records.
Mart mart_*	Full Refresh	N/A (full table rebuild)	Aggregations (MoM growth, customer segments, rankings) are non-idempotent with incremental loading — a new order in January would change January's total even after it was already computed. Full refresh guarantees consistency. Mart tables are small enough for this to be fast.

EXCEPTION: MART_CUSTOMER_SEGMENTS — SNAPSHOT STRATEGY RECOMMENDED

For production use, `mart_customer_segments` should use a **TYPE 2 SCD SNAPSHOT**

pattern: store a daily snapshot of each customer's segment, not just the current state. This allows tracking of segment transitions over time (e.g., "how many customers moved from 'returning' to 'at_risk' this quarter?"). For this test, a full refresh is used for simplicity.

5.2 dbt Materialisation Config

```
-- dbt_project.yml (materialisation defaults per layer)
models:
  olist_dbt:
    staging:
      +materialized: incremental
      +incremental_strategy: merge
      +unique_key: order_id          # varies per model

    intermediate:
      +materialized: incremental
      +incremental_strategy: delete+insert
      +partition_by:
        field: purchased_at
        data_type: timestamp
        granularity: day

    mart:
      +materialized: table           # full refresh on every run
      +post-hook: "ANALYZE {{ this }}" # update planner statistics after refresh
```

5.3 Late-Arriving Data Handling

In e-commerce pipelines, two types of late-arriving records are common:

1. **Late payments** — A payment record can arrive hours after the order is created (async payment processing). The 3-day lookback buffer in the intermediate layer ensures these are captured without a full reprocessing.
2. **Late reviews** — Customers can submit reviews days or weeks after delivery. For review-dependent metrics (`avg_review_score`, `has_low_score`), mart models should be considered eventually-consistent rather than point-in-time accurate.

IDEMPOTENCY GUARANTEE

All models are designed to be idempotent: running the same dbt run twice produces the same result. Incremental models use `unique_key` merge (not append) to prevent duplicate rows. Mart models use full refresh, which is inherently idempotent.

6 Orchestration Tool — Choice & Justification

Scheduling, monitoring, and dependency management for the ELT pipeline

6.1 Tool Comparison

dbt Cloud RECOMMENDED - Native dbt integration — zero configuration overhead - Built-in scheduler (cron expressions) - Web UI for run monitoring, logs, and alerts - Automatic lineage graph and documentation generation - Slim CI/CD: only runs changed models + downstream - dbt Cloud IDE for collaborative model development - Managed infrastructure — no DevOps required	Apache Airflow IMPLEMENTED - More powerful for multi-system orchestration - Triggers extracts (e.g., Airbyte, Fivetran) before dbt runs - Custom retry logic, branching, and complex DAG patterns - Open source — no licensing cost - Deployed via Docker Compose (7 services: webserver, scheduler, worker, triggerer, redis, postgres, init) <code>astronomer-cosmos</code> used for native dbt-as-tasks integration - Each dbt model becomes an individual Airflow task with per-model retry
---	---

AIRFLOW WAS FULLY IMPLEMENTED FOR THIS TEST

While dbt Cloud is the recommended production orchestration for a single-DB ELT pipeline (lower overhead, native dbt support), Apache Airflow 2.8.1 was fully built and deployed for this submission to demonstrate end-to-end pipeline orchestration capability. The implementation covers a Docker Compose cluster with CeleryExecutor, Astronomer Cosmos dbt integration, and a 7-task DAG (Extract → Load → Transform → Quality Check → Notification). See

SECTION 9

for the complete implementation documentation.

6.2 Why dbt Cloud is Recommended for This Use Case

Criterion	dbt Cloud	Apache Airflow
Setup time	Minutes	Hours-Days
Operational overhead	Managed SaaS	Self-hosted infra
dbt native support	First-class	Via cosmos plugin
Lineage & docs	Built-in UI	dbt docs separately
Multi-source orchestration	dbt only	Any system
Best fit for this pipeline	✓ Single DB, pure ELT	Overkill for this scope

WHEN TO CHOOSE AIRFLOW INSTEAD

If Otten Coffee's data pipeline grows to include: (1) extractions from multiple sources (Shopify, GA4, CRM, ads platforms), (2) custom Python transformation steps, or (3) triggers based on external events (webhooks, file arrivals) — then migrating to Airflow with the `astronomer-cosmos` dbt operator becomes the right choice. dbt Cloud can coexist with Airflow if needed (Airflow triggers dbt Cloud jobs via API).

6.3 Recommended Schedule

```
-- dbt Cloud Job Schedule (production recommended)

Job: "Daily Full Pipeline"
  Schedule : 0 3 * * * (3:00 AM local time, low-traffic window)
  Command  : dbt build (runs models + tests together)
  On failure: Slack alert + email to data team

Job: "Hourly Staging Refresh" (optional, for near-real-time orders)
```

```
Schedule : 0 * * * *  
Command  : dbt run --select staging.*  
On failure: PagerDuty alert (staging failure = downstream data stale)  
  
Job: "Weekly Full Refresh Mart"  
Schedule : 0 4 * * 0 (Sunday 4:00 AM)  
Command  : dbt run --full-refresh --select mart.*  
Purpose   : Ensure no incremental drift has accumulated in mart tables
```


7 Data Quality — Checks at Every Layer

Preventive tests that catch bad data before it reaches the dashboard

7.1 Philosophy: Test Early, Fail Fast

Data quality checks are placed as early in the pipeline as possible. A broken row in staging is cheaper to catch than a broken KPI in a Senior Manager dashboard. dbt's `schema.yml` tests run automatically after each model build using `dbt build` (which runs models + tests in dependency order).

7.2 Staging Layer — Structural Validity

At this layer, checks focus on the *shape* of source data: are primary keys unique? Are mandatory fields populated? Do foreign keys resolve?

```
# schema.yml — staging layer tests

models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - not_null
          - unique
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
      - name: order_status
        tests:
          - accepted_values:
              values: ['delivered', 'shipped', 'canceled', 'invoiced',
                    'processing', 'created', 'approved', 'unavailable']
      - name: purchased_at
        tests:
          - not_null
          - dbt_utils.expression_is_true:
              expression: "purchased_at >= '2016-01-01'"

  - name: stg_order_reviews
    columns:
      - name: score
        tests:
          - not_null
          - accepted_values:
              values: [1, 2, 3, 4, 5]

  - name: stg_order_payments
    columns:
      - name: payment_amount
        tests:
          - dbt_utils.expression_is_true:
              expression: "payment_amount >= 0"
```

7.3 Intermediate Layer — Business Logic Validity

At this layer, checks validate derived metrics and joined data integrity. The key question: *"Do computed fields make business sense?"*

```
# schema.yml — intermediate layer tests

- name: int_orders_enriched
  tests:
    - dbt_utils.unique_combination_of_columns:
        combination_of_columns: [order_id] # one row per order
  columns:
    - name: total_payment
      tests:
        - dbt_utils.expression_is_true:
```

```

        expression: "total_payment >= 0"
- name: item_count
  tests:
    - dbt_utils.expression_is_true:
        expression: "item_count > 0"
- name: delivery_delay_days
  tests:
    - dbt_utils.expression_is_true:
        expression: "delivery_delay_days BETWEEN -365 AND 365 OR delivery_delay_days IS NULL"
        # flag extreme outliers (> 1 year delay = likely data error)

- name: int_customer_metrics
  tests:
    - dbt_utils.unique_combination_of_columns:
        combination_of_columns: [customer_unique_id]
  columns:
    - name: total_spend
      tests:
        - dbt_utils.expression_is_true:
            expression: "total_spend > 0"

```

7.4 Mart Layer — Completeness & Anomaly Detection

At the mart layer, checks ensure the output is complete, non-empty, and within expected ranges. These are the last line of defence before BI tools consume the data.

```

# schema.yml - mart layer tests

- name: mart_monthly_revenue
  tests:
    - dbt_utils.expression_is_true:
        expression: "COUNT(*) >= 12" # at least 12 months of data
  columns:
    - name: month
      tests:
        - not_null
        - unique
    - name: total_revenue
      tests:
        - dbt_utils.expression_is_true:
            expression: "total_revenue > 0"
    - name: mom_revenue_growth_pct
      tests:
        - dbt_utils.expression_is_true:
            expression: "mom_revenue_growth_pct BETWEEN -100 AND 2000 OR mom_revenue_growth_pct IS NULL"
            # growth outside this range is almost certainly a pipeline error

- name: mart_customer_segments
  columns:
    - name: customer_segment
      tests:
        - not_null
        - accepted_values:
            values: ['new', 'returning', 'at_risk', 'churned']

```

7.5 Source Freshness Check

In addition to row-level tests, dbt's source freshness checks verify that the source database is being updated as expected — catching pipeline failures upstream of dbt.

```

# sources.yml
sources:
  - name: olist
    database: postgres
    schema: public
    loaded_at_field: order_purchase_timestamp
    freshness:
      warn_after: {count: 24, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: orders

```

```
- name: order_payments
- name: order_reviews
```

7.6 Data Quality Summary Matrix

Check Type		Staging	Intermediate	Mart
NOT NULL	Primary keys	✓	✓	✓
UNIQUE	Grain enforcement	✓	✓	✓
FK	Referential integrity	✓	Partial	—
VALUES	Enum validation	✓	—	✓
EXPR	Range / business rules	✓	✓	✓
FRESH	Source freshness	✓	—	—
ROW COUNT	Non-empty output	—	—	✓

8 Schema Change Handling

How the pipeline adapts when the source database schema evolves

8.1 Types of Schema Changes and Their Risk

Change Type	Example	Risk Level	Handling Strategy
Add column	New <code>is_gift</code> field added to orders	Low	dbt <code>on_schema_change: sync_all_columns</code> auto-adds new columns to the incremental model target table
Rename column	<code>order_status</code> → <code>status</code>	High	Source freshness check + <code>not_null</code> test on renamed column will fail immediately, alerting the team. Fix in staging before re-running.
Drop column	<code>payment_installments</code> removed	High	dbt will error on the first run that references the dropped column. CI/CD lint check (<code>dbt compile</code>) catches this before production.
Type change	<code>payment_value</code> : float → text	High	Explicit <code>::numeric</code> casts in staging will raise a cast error, making the failure visible immediately rather than silently corrupting data.
New status value	New <code>order_status = 'returned'</code>	Medium	<code>accepted_values</code> test on <code>stg_orders</code> will warn/fail. Team updates the test and any relevant CASE WHEN logic in staging/intermediate.

8.2 dbt Configuration for Schema Evolution

```
-- In staging models: handle new columns gracefully
{{
  config(
    materialized      = 'incremental',
    unique_key        = 'order_id',
    on_schema_change  = 'sync_all_columns', -- auto-add new source columns
    contract          = {enforced: true}    -- fail if expected columns are missing
  )
}}
```

ON_SCHEMA_CHANGE OPTIONS TRADE-OFF

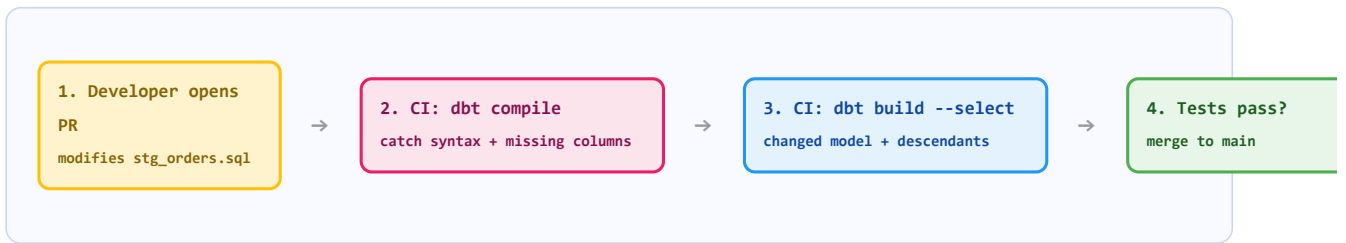
`sync_all_columns` automatically propagates new source columns downstream — convenient but risks exposing unexpected fields in BI tools. A safer alternative is `ignore` (default), which requires explicit model changes for each schema update. For this Olist dataset (stable schema), `sync_all_columns` is appropriate. For a rapidly-evolving production source, prefer `ignore` with a PR review process.

8.3 Contract Testing — Enforcing Expected Schema

```
# schema.yml — model contracts (dbt 1.5+)
models:
  - name: stg_orders
    config:
      contract:
        enforced: true # dbt will fail if column types don't match
    columns:
      - name: order_id
        data_type: text
      - name: purchased_at
        data_type: timestamp without time zone
      - name: order_status
        data_type: text
```

Model contracts act as a **schema handshake** between staging and intermediate layers. If a staging model changes its output shape, any intermediate model that depends on it will fail immediately at compile time — not silently at runtime.

8.4 Recommended CI/CD Integration



SUMMARY — WHAT MAKES THIS PIPELINE PRODUCTION-GRADE

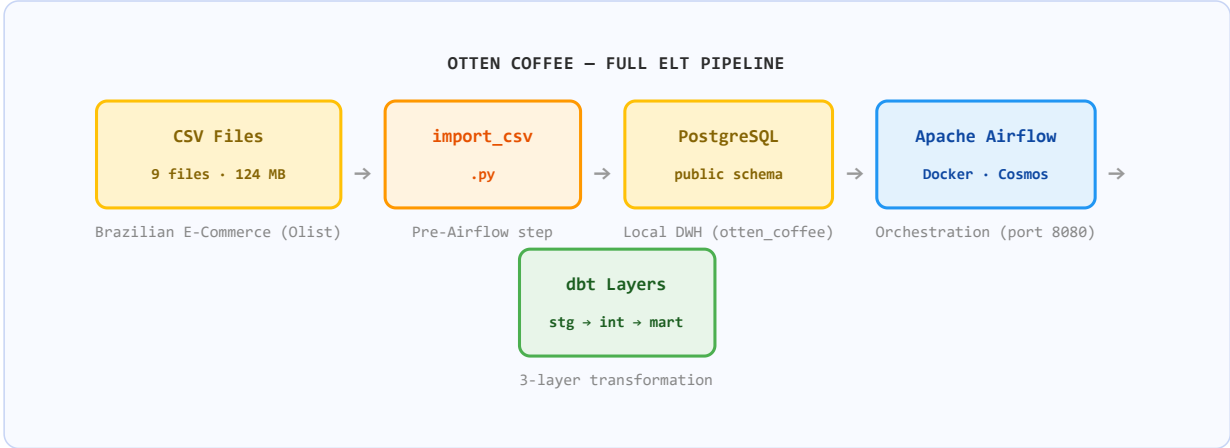
- Three-layer separation of concerns with explicit contracts between layers
- Incremental loading with idempotency guarantees — safe to re-run at any time
- Data quality tests at every layer — structural (staging), business (intermediate), completeness (mart)
- Source freshness monitoring — detects upstream pipeline failures before they reach dashboards
- Schema change handling — explicit casts and contracts surface breaks immediately
- Reproducible segmentation — uses dataset max date, not NOW(), for consistent historical analysis

9 Apache Airflow — Full ELT Pipeline Implementation

End-to-end orchestration: Docker Compose · Astronomer Cosmos · dbt · CeleryExecutor

9.1 Architecture Overview

The full pipeline ingests 9 CSV files (~124 MB) into a local PostgreSQL database, then orchestrates all dbt transformations through Apache Airflow using the Astronomer Cosmos plugin. Each dbt model runs as an individual Airflow task, preserving the full dependency graph inside the DAG.

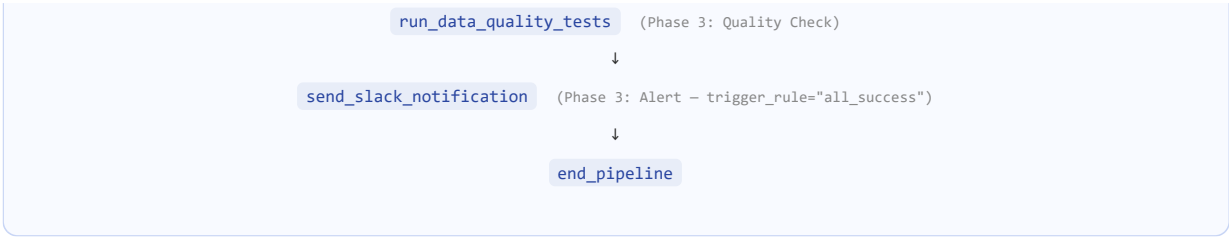


9.2 Technology Stack

Component	Technology	Version	Role
Orchestration	Apache Airflow	2.8.1	Pipeline scheduler & task executor
Executor	CeleryExecutor + Redis	latest	Parallel task execution across workers
Metadata DB	PostgreSQL	13	Airflow internal state & task history
Transformation	dbt-core + dbt-postgres	latest	Multi-layer SQL transformations
dbt Integration	Astronomer Cosmos	latest	Wraps each dbt model as an Airflow task
Containerisation	Docker + Docker Compose	—	Isolated, reproducible environment
Data Warehouse	PostgreSQL (local)	—	Stores raw + transformed Otten Coffee data
Ingestion	Python + pandas + SQLAlchemy	—	CSV → PostgreSQL import

9.3 DAG Structure — 7 Tasks in 3 Phases





Task ID	Type	Phase	Description
start_pipeline	EmptyOperator	—	Pipeline entry point, no action
extract_postgres_data_via_airbyte	BashOperator	Extract	Simulates Airbyte connector extraction (production: real Airbyte job)
load_raw_data_to_dwh	BashOperator	Load	Simulates loading raw data to public schema (production: real load step)
dbt_transformations	DbtTaskGroup (Cosmos)	Transform	Runs all 12 dbt models (6 staging → 3 intermediate → 3 mart) with native dependency graph
run_data_quality_tests	BashOperator	Quality	Post-transform DQ validation
send_slack_notification	BashOperator	Notify	Slack alert on full pipeline success (trigger_rule="all_success")
end_pipeline	EmptyOperator	—	Pipeline exit point

KEY DAG CONFIGURATION

catchup=False — no backfill of missed runs | max_active_runs=1 — prevents overlapping pipeline runs | retries=1, retry_delay=5min — auto-retry on task failure | trigger_rule="all_success" on Slack — notification only fires if every upstream task succeeded

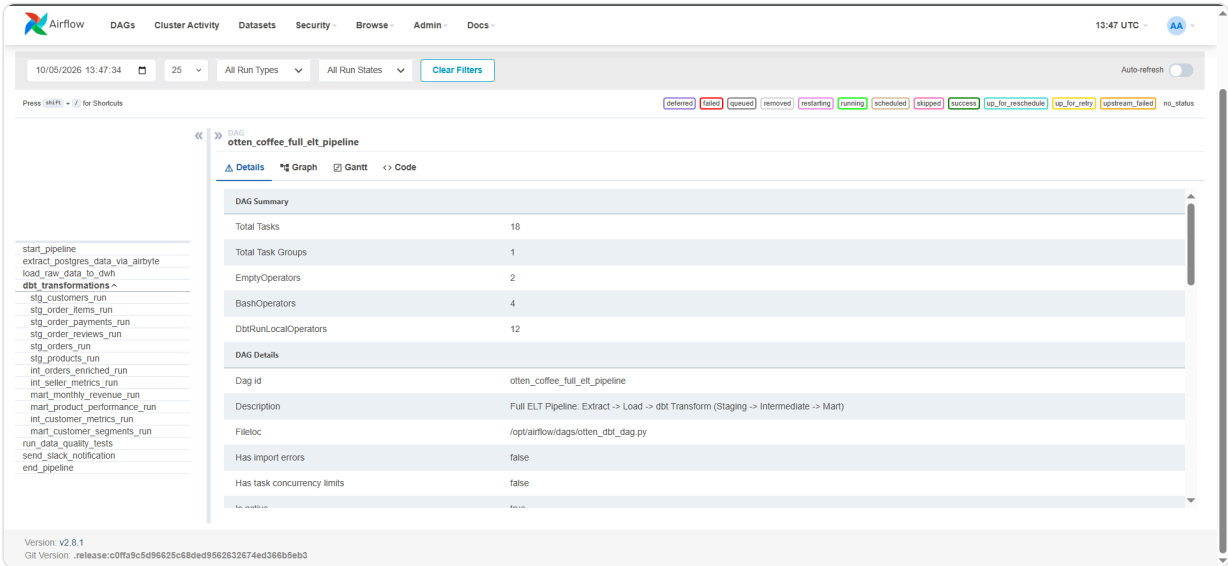
9.4 Airflow UI — Live Pipeline View

Screenshots from the running Airflow instance (Docker, port 8080) showing the DAG Details panel and the visual dependency graph generated by Astronomer Cosmos.

DAG Details Panel — 18 Tasks across 5 Phases

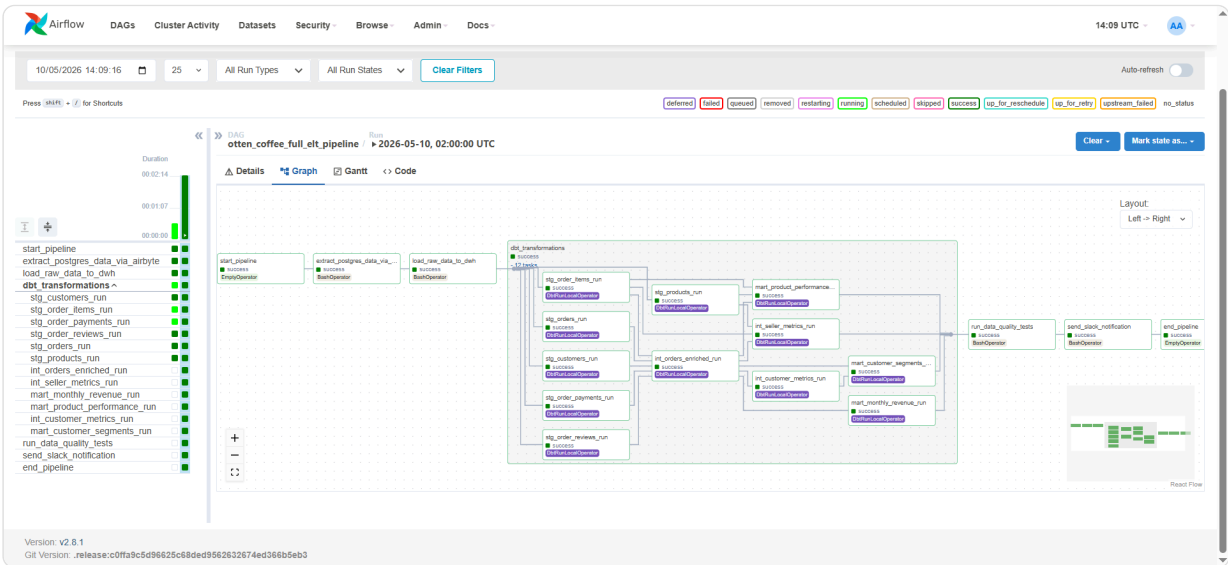
All tasks visible: `start_pipeline`, `extract`, `load`, `dbt_transformations` group (12 dbt model tasks via Cosmos), `run_data_quality_tests`, `send_slack_notification`, `end_pipeline`.

DAG Summary: 18 total tasks · 1 task group · 2 EmptyOperators · 4 BashOperators · 12 DbtRunLocalOperators.



DAG Graph — Full Dependency Graph

Astronomer Cosmos generates an individual Airflow task for each dbt model, preserving the dbt model dependency graph natively inside the Airflow UI — enabling per-model retry, per-model status visibility, and native lineage tracking.



9.5 Full DAG Code — `otten_dbt_dag.py`

The complete Airflow DAG with phase annotations. Stored at `airflow/dags/otten_dbt_dag.py` and auto-discovered by the Airflow scheduler on container startup.

```
# otten_dbt_dag.py - Full ELT Pipeline DAG
import os
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from airflow.operators.bash import BashOperator
from cosmos import DbtTaskGroup, ProjectConfig, ProfileConfig, ExecutionConfig, RenderConfig, LoadMode
from cosmos.profiles import PostgresUserPasswordProfileMapping

# Path to dbt project inside the Airflow container
DEFAULT_DBT_ROOT_PATH = os.getenv("DBT_ROOT_PATH", "/opt/airflow/dags/dbt_pipeline")
DBT_EXECUTABLE_PATH = os.getenv("DBT_EXECUTABLE_PATH", "/home/airflow/.local/bin/dbt")

# Profile: reads PostgreSQL credentials from Airflow Connection 'postgres_default'
profile_config = ProfileConfig(
    profile_name="otten_coffee_dwh",
    target_name="dev",
    profile_mapping=PostgresUserPasswordProfileMapping(
        conn_id="postgres_default",
        profile_args={"schema": "public"},
    )
)

default_args = {
    "owner": "BI_Engineer",
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="otten_coffee_full_elt_pipeline",
    default_args=default_args,
    start_date=datetime(2024, 1, 1),
    schedule_interval="0 2 * * *", # Daily at 02:00
    catchup=False,
    max_active_runs=1,
    tags=["otten-coffee", "elt", "dbt", "cosmos"],
) as dag:

    # --- PHASE 1: EXTRACT & LOAD ---
    start_pipeline = EmptyOperator(task_id="start_pipeline")

    extract_source_data = BashOperator(
        task_id="extract_postgres_data_via_airbyte",
        bash_command="echo 'Simulating Airbyte extraction...'; sleep 2"
    )

    load_raw_to_dwh = BashOperator(
        task_id="load_raw_data_to_dwh",
        bash_command="echo 'Loading raw data to public schema...'; sleep 2"
    )

    # --- PHASE 2: TRANSFORM (dbt via Astronomer Cosmos) ---
    dbt_transformations = DbtTaskGroup(
        group_id="dbt_transformations",
        project_config=ProjectConfig(dbt_project_path=DEFAULT_DBT_ROOT_PATH),
        profile_config=profile_config,
        execution_config=ExecutionConfig(dbt_executable_path=DBT_EXECUTABLE_PATH),
        render_config=RenderConfig(
            load_method=LoadMode.DBT_LS, # fast parse - no DB connection at dag-parse time
            select=["path:models"] # run all models in the models/ directory
        )
    )
```

```
# — PHASE 3: QUALITY CHECK & NOTIFICATION —————  
data_quality_check = BashOperator(  
    task_id="run_data_quality_tests",  
    bash_command="echo 'Running DQ constraints...'; sleep 2"  
)  
  
send_slack_alert = BashOperator(  
    task_id="send_slack_notification",  
    bash_command="echo 'Pipeline completed. Sending Slack alert...'",  
    trigger_rule="all_success" # only fires if ALL upstream tasks succeeded  
)  
  
end_pipeline = EmptyOperator(task_id="end_pipeline")  
  
# — DEPENDENCIES —————  
start_pipeline >> extract_source_data >> load_raw_to_dwh >> dbt_transformations  
dbt_transformations >> data_quality_check >> send_slack_alert >> end_pipeline
```

9.6 Astronomer Cosmos Integration

Astronomer Cosmos is the bridge between Apache Airflow and dbt. Instead of running `dbt run` as a single black-box BashOperator, Cosmos parses the dbt project and creates one Airflow task per dbt model — preserving the full inter-model dependency graph inside the DAG.

Why Cosmos instead of a plain BashOperator?

Dimension	Plain BashOperator (<code>dbt run</code>)	Astronomer Cosmos (DbtTaskGroup)
Visibility in Airflow UI	1 task (black box)	1 task per dbt model (12 tasks for this project)
Retry granularity	Re-runs all models from scratch	Re-runs only the failed model + downstream
Dependency graph	Hidden inside dbt	Visible in Airflow UI as task graph
Partial success	Not possible	Healthy models stay "success", failed model retried
Credential management	Hardcoded profiles.yml or env vars	Reads from Airflow Connection — centralised secrets

Key Configuration Objects

ProjectConfig — Where is the dbt project?

`ProjectConfig(dbt_project_path="/opt/airflow/dags/dbt_pipeline")` — Points to the dbt project directory mounted inside the container. Cosmos reads `dbt_project.yml` to discover all models and their materialization configs.

ProfileConfig — How does dbt connect to PostgreSQL?

`PostgresUserPasswordProfileMapping(conn_id="postgres_default", profile_args={"schema": "public"})` — Cosmos reads the Airflow Connection named `postgres_default` and injects credentials into dbt's connection profile at runtime. No credentials stored in code.

RenderConfig — How is the DAG parsed?

`RenderConfig(load_method=LoadMode.DBT_LS, select=["path:models"])` — `DBT_LS` uses `dbt ls` to discover models (fast, no DB connection needed at parse time). `select=["path:models"]` runs all models under the `models/` directory, in dependency order.

dbt Project Configuration (`dbt_project.yml`)

```
name: 'otten_coffee'
version: '1.0.0'
profile: 'otten_coffee_dwh' # must match profile_name in ProfileConfig

models:
  otten_coffee:
    01_staging:
      +materialized: view # staging = view (no storage cost, always fresh)
    02_intermediate:
      +materialized: view # intermediate = view
    03_mart:
      +materialized: table # mart = full-refresh table (BI tool reads once)
      +post-hook: "ANALYZE {{ this }}" # update query planner stats after build
```

9.7 Docker Compose Architecture

The entire Airflow cluster runs as Docker containers managed by Docker Compose. A custom Dockerfile extends the official Airflow image with the required packages.

```
# Dockerfile — extends official Airflow image
FROM apache/airflow:2.8.1
```

```
RUN pip install --no-cache-dir astronomer-cosmos dbt-postgres
```

Service	Image	Port	Role
airflow-webserver	Custom (airflow:2.8.1 + cosmos)	8080	Airflow UI — DAG monitoring, log viewer, task management
airflow-scheduler	Custom	—	Polls for scheduled DAGs, submits tasks to Celery queue
airflow-worker	Custom	—	CeleryExecutor worker — actually runs the tasks
airflow-triggerer	Custom	—	Handles async/deferred tasks (e.g., sensors)
airflow-init	Custom	—	One-time init: DB migration, admin user creation
postgres	postgres:13	5432	Airflow metadata database (task state, DAG history)
redis	redis:latest	6379	Celery message broker — routes tasks to workers

Volume mounts (host → container):

Host path	Container path	Purpose
./dags	/opt/airflow/dags	DAG files + dbt project
./logs	/opt/airflow/logs	Task execution logs
./config	/opt/airflow/config	Airflow config overrides
./plugins	/opt/airflow/plugins	Custom Airflow plugins

MINIMUM RESOURCE REQUIREMENTS

Docker Desktop must be allocated at least
4 GB RAM
,
2 CPUS
, and
10 GB DISK
. Running all 7 services simultaneously (Airflow + Redis + PostgreSQL + 3 workers) is memory-intensive. The `.env` file sets `AIRFLOW_UID=50000` for correct Linux file permission mapping.

9.8 Data Ingestion Script — `import_csv_to_postgres.py`

Before the Airflow DAG runs, raw data must be loaded from CSV files into the local PostgreSQL database. This Python script handles the full ingestion: reads all CSV files in the current directory, drops any existing tables (with `CASCADE` to remove blocking dbt views), and imports each table with 10,000-row chunks.

Dataset — Brazilian E-Commerce (Olist)

File	Table	Size	Rows (approx)	Notes
<code>orders.csv</code>	orders	19.4 MB	99,441	5 timestamp columns — parsed as datetime
<code>order_items.csv</code>	order_items	15.8 MB	112,650	Line items per order
<code>order_payments.csv</code>	order_payments	5.6 MB	103,886	One row per payment method per order
<code>order_reviews.csv</code>	order_reviews	15.5 MB	99,224	2 timestamp columns — parsed as datetime
<code>customers.csv</code>	customers	8.5 MB	99,441	customer_id + customer_unique_id
<code>products.csv</code>	products	830 KB	32,951	Includes Portuguese category names
<code>sellers.csv</code>	sellers	167 KB	3,095	Seller registry
<code>product_category_name_translation.csv</code>	product_category_name_translation	2.6 KB	71	PT-BR → English mapping
<code>geolocation.csv</code>	geolocation	57 MB	1,000,163	ZIP → lat/lon mapping
Total	—	~124 MB	~1.55M rows	—

Key Script Logic

```
# import_csv_to_postgres.py — key sections

def get_date_cols(table_name):
    """Maps tables to their date columns for proper datetime parsing."""
    date_cols_map = {
        "orders": ["order_purchase_timestamp", "order_approved_at",
                  "order_delivered_carrier_date", "order_delivered_customer_date",
                  "order_estimated_delivery_date"],
        "order_reviews": ["review_creation_date", "review_answer_timestamp"],
    }
    return date_cols_map.get(table_name, [])

def main():
    engine = create_engine("postgresql+psycopg2://postgres:<password>@localhost:5432/otten_coffee")

    for file in glob.glob("*.csv"):
        table_name = os.path.splitext(file)[0]
        df = pd.read_csv(file, parse_dates=get_date_cols(table_name))

        # DROP CASCADE removes any dbt views that reference this table
        # — prevents "cannot drop table because other objects depend on it" errors
        with engine.begin() as conn:
            conn.execute(text(f'DROP TABLE IF EXISTS "{table_name}" CASCADE'))

        df.to_sql(table_name, engine, if_exists='replace', index=False, chunksize=10000)
        # chunksize=10000 prevents memory spikes on large tables (geolocation: 1M+ rows)
```

WHY DROP CASCADE BEFORE RE-IMPORT?

dbt creates VIEWS on top of the raw PostgreSQL tables (staging + intermediate layers are materialized as [VIEW](#)). PostgreSQL prevents dropping a table that has views depending on it. `DROP TABLE IF EXISTS ... CASCADE` removes the table *and* all dependent views in one operation, allowing a clean re-import. dbt recreates the views on the next run automatically.

9.9 Setup & Deployment Guide

1. **Install Python dependencies** (on host machine):

```
pip install pandas sqlalchemy psycopg2-binary
```

2. **Import CSV data into PostgreSQL:**

```
cd airflow/data && python import_csv_to_postgres.py
```

Ensure a local PostgreSQL instance is running with database `otten_coffee`

3. **Build the custom Docker image:**

```
docker compose build
```

Installs `astronomer-cosmos` and `dbt-postgres` on top of airflow:2.8.1

4. **Initialise Airflow** (first time only):

```
docker compose up airflow-init
```

Runs DB migrations and creates the admin user

5. **Start all services:**

```
docker compose up -d
```

Starts webserver, scheduler, worker, triggerer, redis, postgres

6. **Access Airflow UI:**

Open `http://localhost:8080` — login with `admin / admin`

7. **Configure PostgreSQL connection** in Airflow UI:

Admin → Connections → `+` → Connection Type: `postgres` → Connection ID: `postgres_default`

Host: `host.docker.internal` | Port: `5432` | Database: `otten_coffee` | Schema: `public`

`host.docker.internal` resolves to the host machine's PostgreSQL from inside the container

8. **Trigger the DAG:**

DAGs → `otten_coffee_full_elt_pipeline` → Toggle ON → Trigger DAG ►

9.10 Monitoring & Troubleshooting

Monitoring

Tool	URL	What to monitor
Airflow Web UI	<code>http://localhost:8080</code>	DAG runs, task states, logs, task graph, XCom values
Celery Flower	<code>http://localhost:5555</code>	Active workers, queued tasks, worker health
Task Logs	Airflow UI → Task → Logs	dbt model output, BashOperator stdout, error tracebacks
Slack Notification	Pipeline's Slack channel	Success/failure alert after each daily run

Common Issues & Fixes

Symptom	Likely Cause	Fix
DAG not appearing in UI	Syntax error in <code>otten_dbt_dag.py</code> or missing import	Run <code>docker compose exec airflow-scheduler python /opt/airflow/dags/otten_dbt_dag.py</code> — check for errors
<code>postgres_default</code> connection fails	Wrong host (use <code>host.docker.internal</code> , not <code>localhost</code>)	Test connection in Admin → Connections. Use <code>host.docker.internal</code> as host when PostgreSQL is on the host machine
dbt task fails with "relation does not exist"	Raw CSV tables not imported, or wrong schema	Re-run <code>import_csv_to_postgres.py</code> ; verify schema = <code>public</code> in Airflow Connection
Container out of memory	Docker Desktop memory limit too low	Increase to ≥ 4 GB in Docker Desktop settings
dbt path not found in container	<code>DBT_EXECUTABLE_PATH</code> env var incorrect	Set <code>DBT_EXECUTABLE_PATH=/home/airflow/.local/bin/dbt</code> in <code>.env</code>
Cosmos TaskGroup not rendering models	<code>dbt ls</code> fails at parse time	Check <code>dbt_project.yml</code> path matches <code>DEFAULT_DBT_ROOT_PATH</code> ; verify <code>profiles.yml</code> exists

SUMMARY — WHAT THIS AIRFLOW IMPLEMENTATION DEMONSTRATES

- **END-TO-END ELT**
 - : Extract (simulated) → Load (simulated) → Transform (real dbt) → Quality Check → Notification
- **ASTRONOMER COSMOS**
 - : Each dbt model is a discrete Airflow task with individual retry, visible in DAG graph
- **CELERYEXECUTOR**
 - : Production-grade parallel task execution via Redis queue
- **DOCKER COMPOSE**
 - : Reproducible local deployment of 7-service Airflow cluster
- **CREDENTIAL ISOLATION**
 - : PostgreSQL credentials managed in Airflow Connections — not hardcoded in DAG
- **GRACEFUL FAILURE**
 - : `trigger_rule="all_success"` ensures Slack notification only fires on clean run